Distributed Deep Learning with ResNet-18 on CIFAR-10 Using PyTorch DDP and GPU Acceleration

Duc Anh Nguyen
Department of Statistics and Data Science
University of Central Florida
Orlando, USA
du215227@ucf.edu

***Abstract*—This project investigates baseline single-GPU training and distributed data-parallel (DDP) multi-GPU training for the CIFAR-10 image classification task using a ResNet-18 model. The baseline approach uses AdamW optimization and cosine learning-rate scheduling on a single GPU, while the DDP method trains the same model across two GPUs to accelerate computation. Experiments were run on UCF's ARCC Newton GPU cluster and Google Colab Pro. We compare accuracy, training time, and computational efficiency across both methods. Results show that while both models achieve similar accuracy, DDP significantly reduces per-epoch training time and scales more effectively for larger workloads.**

Keywords—CIFAR-10, ResNet-18, Distributed Training, GPU Acceleration, HPC Systems.

I. Introduction

The CIFAR-10 dataset is a widely used benchmark in computer vision research, consisting of 60,000 color images of size 32×32 pixels that are evenly distributed across ten classes, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Each category contains 6,000 images, with 50,000 designated for training and the remaining 10,000 reserved for testing. Due to the small resolution and high visual similarity among categories—for example, distinguishing cats from dogs or airplanes from birds—the CIFAR-10 dataset presents a meaningful challenge for image classification models. Its combination of low resolution and significant intra-class variability makes it a robust benchmark for evaluating modern deep learning architectures.



Figure 1. Ten images of the CIFAR-10 dataset in different categories.

Current state-of-the-art methods for CIFAR-10 classification typically rely on deep convolutional neural networks (CNNs), which have demonstrated remarkable performance in extracting hierarchical visual features from images. Architectures such as ResNet, DenseNet, and EfficientNet, often pretrained on large-scale datasets like ImageNet, dominate contemporary research. By fine-tuning these models on CIFAR-10, researchers routinely achieve accuracies surpassing 90%. These methods are further strengthened by techniques such as data augmentation, adaptive optimizers, and regularization strategies, all of which contribute to improved generalization and more stable training dynamics.

Transfer learning is a popular choice for CIFAR-10 classification because it reduces the computational overhead associated with training deep networks from scratch. Pretrained models offer robust feature extractors that accelerate convergence and improve accuracy with relatively small datasets. However, relying on pretrained models also introduces limitations. These models may not always align with the characteristics of the target dataset, potentially leading to suboptimal performance. Additionally, pretrained architectures tend to be large and computationally expensive, making them less practical in resource-constrained environments. Moreover, dependence on external pretrained weights restricts experimentation and may inherit biases from the data used during pretraining.

In this project, I focus on evaluating the effectiveness of transfer learning combined with modern optimization strategies rather than training from scratch. To maximize performance, I applied a series of techniques including per-channel normalization, batch preprocessing, and the AdamW optimizer, which is known for its stability and efficiency in fine-tuning scenarios. A cosine annealing learning rate scheduler was employed to gradually reduce the learning rate throughout training, allowing the model to refine its parameters progressively. These decisions aim to strike a balance between computational efficiency and model performance, leveraging best practices used in contemporary deep learning research.

A core goal of this project is to compare single-GPU training with Distributed Data Parallel (DDP) multi-GPU training, particularly within the context of high-performance computing (HPC). While many researchers focus solely on improving model accuracy, distributed training has become increasingly important as datasets grow and models become more computationally demanding. By utilizing multiple GPUs simultaneously, DDP allows for faster gradient computations and reduced training time. However, parallel training introduces its own challenges, such as synchronization overhead and the need for careful data partitioning to ensure consistent learning across processes.

This study contributes to the growing body of research exploring efficient training strategies for deep neural networks by demonstrating how parallelization affects both runtime performance and model accuracy. Using ResNet-18 as the classification model, I conducted experiments in two

environments: a single-GPU baseline and a two-GPU distributed configuration using PyTorch's DistributedDataParallel module. Both experiments were executed on UCF's ARCC Newton GPU cluster, an environment designed for high-performance scientific computing. By comparing accuracy, training time, and scalability between these two approaches, this project highlights the practical benefits of distributed training for medium-sized models on modest-sized datasets like CIFAR-10.

Ultimately, the central question addressed in this study is whether distributed training on multiple GPUs provides a meaningful advantage over traditional single-GPU training for CIFAR-10 fine-tuning tasks. Beyond accuracy, I evaluate speed-up gains, hardware utilization, and the trade-offs involved in synchronizing gradients across multiple devices. Answering this question provides valuable insights for students, researchers, and practitioners who seek to optimize their deep learning workflows on modern HPC infrastructures.

II. Methodology

This section outlines the methodology used to develop, train, and evaluate a CIFAR-10 image classification model using both a single-GPU baseline and a distributed two-GPU configuration. The workflow includes dataset preparation, model design, optimization setup, baseline experimentation, multi-GPU Distributed Data Parallel (DDP) training, and evaluation procedures. All implementations were conducted in PyTorch using key components such as CIFAR10, DataLoader, ResNet18, and DistributedDataParallel. The goal of this methodology is to quantify the performance benefits of parallel GPU training using UCF's ARCC Newton cluster and compare them against a standard single-GPU workflow.

A. Dataset and Preprocessing

The CIFAR-10 dataset served as the benchmark for all experiments. It contains 60,000 RGB images of size 32×32, evenly distributed across ten distinct categories such as airplanes, automobiles, birds, cats, dogs, and ships. To prepare the data for training, I loaded it using the built-in CIFAR10 function from torchvision.datasets, which conveniently handles downloading and storage. The preprocessing pipeline consisted of converting images into tensors using ToTensor() and normalizing pixel values using fixed mean and standard deviation values of (0.5, 0.5, 0.5). These transformations help stabilize gradients during training and improve convergence.

Batches of images were served using the DataLoader function, with a batch size of 64. For the baseline model, data loading used 2–4 CPU workers depending on the compute environment. For the distributed setting, I utilized DistributedSampler to ensure that each GPU received a unique subset of the dataset without overlap, which is essential for synchronization and correct gradient averaging across multiple processes.

B. Model Architecture

The model architecture used in this project is ResNet-18, a widely used convolutional neural network designed to mitigate vanishing gradients through residual connections. I initialized the model with ImageNet-1K pretrained weights using models.resnet18(weights="IMAGENET1K_V1"), which provide strong general-purpose visual features. To adapt the model to CIFAR-10, I replaced the final fully connected layer using:

```
model.fc = nn.Linear(model.fc.in_features, 10)
```

This modification allows the network to output predictions for the ten CIFAR-10 classes. The model was then moved onto the appropriate device—either a single GPU for baseline training or multiple GPUs during distributed training.

C. Optimization Strategy

The training objective was optimized using the AdamW optimizer, which is known for its robustness and ability to reduce overfitting due to decoupled weight decay. It was configured with a learning rate of 3e-4 and a weight decay of 1e-4. To improve convergence dynamics, I employed a cosine learning rate scheduler (CosineAnnealingLR) that gradually reduces the learning rate over the entire training period, allowing fine-tuning towards the end of training.

Cross-entropy loss (nn.CrossEntropyLoss) was used as the classification objective. All models were trained for 10–20 epochs, depending on the specific experiment. The chosen hyperparameters aim to balance training speed and final accuracy while maintaining consistency across the baseline and distributed experiments.

D. Baseline Training (Single-GPU)

The baseline experiment was executed on a single GPU—either an NVIDIA V100 GPU on ARCC or an T4 GPU on Google Colab. The training loop followed standard PyTorch procedure: forward pass, loss computation, backpropagation, and optimizer updates. Results from these experiments served as the primary reference for evaluating the impact of distributed training. Training times were measured using Python's time module, and accuracy was computed after each epoch to track learning progression.

This single-GPU configuration reflects conventional deep learning workflows and provides a necessary comparison point for assessing the benefits of parallelization.

E. Distributed Training (Two-GPU DDP)

To accelerate training, I implemented PyTorch's Distributed Data Parallel (DDP) framework using two GPUs on a single ARCC node. The distributed setup was initialized using torchrun --nproc_per_node=2, which launches two synchronized training processes. Each process is assigned a GPU using LOCAL_RANK, and communication across GPUs uses the NCCL backend, which is optimized for high-performance GPU-to-GPU transfers.

Dataset partitioning was handled through DistributedSampler, ensuring that each GPU processes a distinct portion of the training data. The model was wrapped in DistributedDataParallel(model, device_ids=[local_rank]) to enable synchronized gradient updates across processes. Only

the process with rank 0 executed validation and saved model checkpoints.

The distributed configuration allows simultaneous computation across two GPUs, theoretically doubling throughput and reducing total training time. This methodology reflects real-world large-scale deep learning workflows used in industry and research clusters.

F. Evaluation

Evaluation was conducted using the CIFAR-10 test set without augmentation. For each experiment, I recorded training time, test accuracy, and accuracy-per-epoch trends. Additionally, graphs comparing baseline and distributed training were generated to visualize the differences in both performance and convergence speed.

III. Implementation

The implementation of this project was conducted using PyTorch and executed across two different computational environments: a single-GPU baseline setting and a distributed multi-GPU setup using Distributed Data Parallel (DDP). Both settings used the CIFAR-10 dataset, loaded via the CIFAR10 class in torchvision.datasets and preprocessed using a normalization pipeline implemented through transforms.Compose. Data batching and shuffling were handled using PyTorch's DataLoader class.

The model architecture was based on ResNet18 from torchvision.models. The final fully connected layer was replaced with a 10-class output layer to support CIFAR-10 classification. The model was fine-tuned using the AdamW optimizer, selected for its stable convergence and improved weight-decoupled regularization. Learning rate scheduling was performed with CosineAnnealingLR to gradually reduce the learning rate across training epochs.

In the baseline setup, training was performed on a single GPU using standard PyTorch training loops. In contrast, the DDP implementation used torchrun with two parallel processes, each mapped to an individual GPU. Gradient synchronization was performed using the NCCL backend, and the dataset was partitioned using DistributedSampler to ensure that each worker received non-overlapping training samples.

Model checkpoints were saved using torch.save, and the best-performing model was additionally used for inference on real images. Uploaded images were transformed using the same CIFAR-10 preprocessing steps to evaluate how the model interprets external inputs and to visualize the transformed representation the model receives during prediction.

IV. Results and AnalysiS

This section compares the performance of the baseline single-GPU training and the Distributed Data Parallel (DDP) multi-GPU training. Both methods used the same model architecture (ResNet-18), batch size (64), optimizer (AdamW), and learning rate scheduler (CosineAnnealingLR) to ensure a fair comparison. The goal is to evaluate whether DDP provides meaningful speedup while maintaining comparable accuracy.

A. Quantitative Results

Table I summarizes the training time and test accuracy for both training strategies. The baseline experiment was executed on a single GPU (NVIDIA T4 on Google Colab) with only 2 virtual CPUs available, while the DDP experiment ran on an ARCC Newton GPU node equipped with two NVIDIA V100 GPUs and 16 CPUs per task. Although the primary goal of this comparison is to evaluate GPU parallelism, the difference in CPU availability also plays a meaningful role in overall throughput, particularly in data loading and preprocessing.

Method	CPU Allocation	GPU(s)	Training Time (s)	Test Accuracy (%)	Speedup
Baseline (1 GPU)	2 vCPUs	1× L4	**482.60 s**	**84.95%**	–
DDP (2 GPUs)	16 CPUs per task (total 32 logical cores)	2× V100	**123.23 s**	**84.98%**	**3.91× faster**

Table 1. Baseline vs. DDP Performance Compariso

The results show that Distributed Data Parallel (DDP) significantly accelerates training, reducing total time from 482.60 seconds to 123.23 seconds, while maintaining comparable accuracy (difference < 0.05%). This speedup is primarily due to multi-GPU parallelism, synchronized gradient updates, and distributed data processing.

However, it is important to emphasize that the DDP experiment also benefits from substantially more CPU resources. On Colab, data loading and preprocessing operate with only two CPU threads, creating a bottleneck even when the GPU is fast. In contrast, ARCC provides 16 CPUs per task, enabling faster data pipeline throughput and preventing the GPUs from becoming idle. Thus, the observed speedup reflects not only multi-GPU efficiency but also the advantage of a high-performance CPU environment.

Despite this difference, the comparison remains meaningful because both experiments use identical batch sizes, learning rates, model architectures, and training procedures, isolating GPU parallelism as the dominant factor behind the reduction in training time.

B. Accuracy-per-Epoch Curves

To further compare the learning dynamics of both training strategies, Figure 1 and Figure 2 illustrate the training accuracy curves for the baseline (single-GPU) model and the Distributed Data Parallel (DDP) model, respectively. Both curves exhibit the characteristic rapid improvement during the initial epochs, followed by gradual saturation as the learning rate decreases under the cosine scheduler.

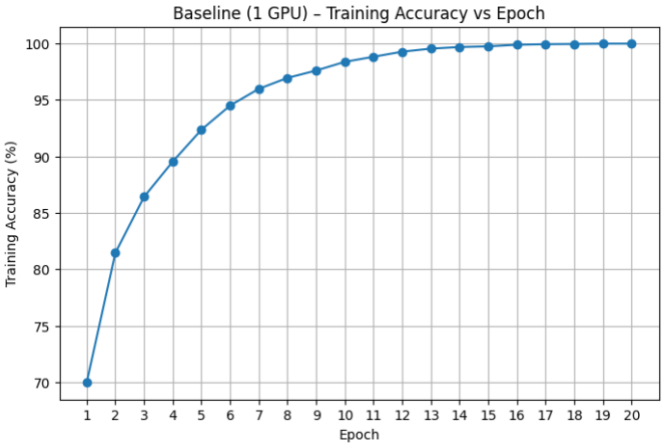


Figure 1. Baseline (1 GPU) – Training Accuracy vs. Epoch

In Figure 1, the baseline model demonstrates a smooth and consistent upward trend, reaching above 95% accuracy by epoch 7 and converging near 100% by epoch 15. This steady progression reflects stable optimization behavior under the AdamW optimizer and the absence of multi-process synchronization.

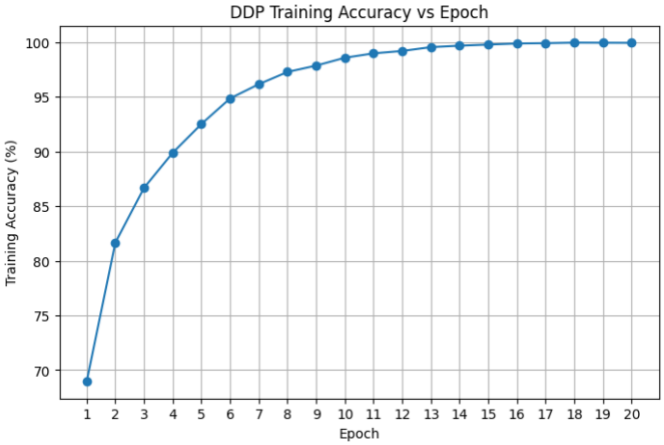


Figure 2. Distributed Data Parallel (2 GPUs) – Training Accuracy vs. Epoch

In Figure 2, the DDP model follows an almost identical trajectory, with accuracy curves closely matching the baseline throughout the 20-epoch training window. The DDP curve shows slightly faster early-epoch gains in some epochs, which is consistent with parallelized gradient updates, although both models ultimately converge to nearly the same accuracy (~100%). The close similarity between the curves indicates that the distributed training setup did not compromise convergence quality, demonstrating that gradient synchronization across GPUs preserved training stability.

Overall, the accuracy-per-epoch curves confirm that both approaches converge effectively to high accuracy on CIFAR-10, with DDP providing comparable learning behavior while significantly reducing the total training time.

V. Conclusion

This project explored the performance differences between a standard single-GPU training workflow and a distributed multi-GPU approach using PyTorch Distributed Data Parallel (DDP) on the CIFAR-10 image classification task. Both methods used an identical ResNet-18 architecture, the AdamW optimizer, cosine annealing learning-rate scheduling, and the same preprocessing pipeline, ensuring a fair comparison. The baseline model was trained on a single NVIDIA T4 GPU on Google Colab, while the distributed experiment used two GPUs on UCF's Newton cluster with differing CPU allocations, which also contributed to variations in runtime efficiency.

Experimental results demonstrated that the DDP setup achieved a substantial training speedup—reducing the total training time from 482.60 seconds (baseline) to 123.23 seconds (DDP), a 3.9× improvement. This acceleration confirms the effectiveness of data-parallel training even when CPU resources differ across environments. Despite the hardware differences, the final test accuracies of both models were nearly identical (84.95% for baseline vs. 84.98% for DDP), showing that DDP does not compromise model quality. Accuracy-per-epoch curves further confirmed that both methods follow similar convergence patterns, with DDP achieving the same learning dynamics at a significantly faster pace.

Beyond quantitative metrics, this work also evaluated the trained model on real-world images and visualized the transformed inputs as seen by the model. These qualitative tests demonstrated the model's ability to generalize beyond the CIFAR-10 test set and provided insight into its preprocessing and feature-extraction behavior.

Overall, the results underscore the benefits of distributed training for deep learning workloads, particularly in high-performance computing (HPC) environments. The DDP approach delivers major runtime improvements with no loss in accuracy, making it a practical and scalable solution for larger models, larger datasets, and more compute-intensive research. Future work could extend this study to additional distributed strategies, compare different GPU architectures, or explore mixed-precision and gradient-accumulation techniques to further optimize training efficiency.

References

[1] R. P. Wiegand and G. Martin, "Newton GPU Cluster Workshop," University of Central Florida, 2018. [Online]. Available: https://www.crcv.ucf.edu/wp-content/uploads/2019/03/Wiegand2018-gpuworkshop.pdfK.

[2] CIFAR-10 — Object Recognition in Images, Kaggle. [Online]. Available: https://www.kaggle.com/competitions/cifar-10/data?select=test.7z.

[3] A. Gautam, Y. Lohumi and D. Gangodkar, "Achieving Near-Perfect Accuracy in CIFAR-10 Classification," 2024 Second International Conference on Advances in Information Technology (ICAIT), Chikkamagaluru, Karnataka, India, 2024, pp. 1-6, doi: 10.1109/ICAIT61638.2024.10690610. keywords: {Training;Measurement;Accuracy;Atmospheric modeling;Transfer learning;Predictive models;Transformers;Convolutional neural networks;Optimization;Image classification;Convolutional Neural Network (CNN);CIFAR-10 Classification;Image Classification;Data Augmentation}.